

12 - Arrays y Objetos

Introducción a los Arrays en JavaScript

Un array (o matriz) en JavaScript es una estructura de datos que permite almacenar múltiples valores en una sola variable. Imagina que es como una lista o una colección de cosas. Por ejemplo, si quieres almacenar una lista de tus colores favoritos o los días de la semana, un array es ideal para ello.

Los arrays son muy versátiles y nos permiten trabajar de manera eficiente con grandes cantidades de información. Además, los valores dentro de un array pueden ser de cualquier tipo: números, cadenas de texto, booleanos, objetos, etc. Lo mejor es que no tienen que ser todos del mismo tipo.

Sintaxis Básica de un Array

Crear un array es muy sencillo. Lo haces utilizando corchetes `[]` y separando los valores por comas. Aquí tienes un ejemplo básico:

```
let colores = ["rojo", "azul", "verde"];
```

En este ejemplo, el array `colores` contiene tres cadenas de texto: "rojo", "azul" y "verde".

Acceder a Elementos del Array

Cada elemento dentro de un array tiene una posición o índice. Los índices empiezan desde 0, lo que significa que el primer elemento tiene el índice 0, el

segundo el índice 1, y así sucesivamente. Si quieres acceder a un elemento específico, lo haces utilizando corchetes y el índice del elemento:

```
console.log(colores[0]); // "rojo"  
console.log(colores[2]); // "verde"
```

En este ejemplo, `colores[0]` accede al primer elemento del array, que es "rojo", y `colores[2]` accede al tercer elemento, que es "verde".

Modificar un Elemento del Array

Los arrays también son mutables, lo que significa que puedes cambiar sus elementos después de haberlos creado. Aquí tienes un ejemplo:

```
colores[1] = "amarillo";  
console.log(colores); // ["rojo", "amarillo", "verde"]
```

Ahora, el segundo elemento del array `colores` ha cambiado de "azul" a "amarillo".

Métodos Comunes para Manipular Arrays

JavaScript proporciona muchos métodos útiles para trabajar con arrays. Vamos a ver algunos de los métodos más comunes, explicados de manera clara y sencilla.

1. `push()` – Añadir un elemento al final del array

El método `push()` se utiliza para añadir uno o más elementos al final de un array. Veamos un ejemplo:

```
let frutas = ["manzana", "banana"];  
frutas.push("naranja");
```

```
console.log(frutas); // ["manzana", "banana", "naranja"]
```

En este caso, hemos añadido "naranja" al final del array frutas.

2. pop() – Eliminar el último elemento del array

El método pop() elimina el último elemento de un array y lo devuelve. Si necesitas quitar algo del final, este es el método adecuado:

```
let frutas = ["manzana", "banana", "naranja"];  
let ultimaFruta = frutas.pop();  
  
console.log(frutas);           // ["manzana", "banana"]  
console.log(ultimaFruta);      // "naranja"
```

Aquí, hemos eliminado "naranja" del array y lo hemos almacenado en la variable ultimaFruta.

3. shift() – Eliminar el primer elemento del array

Si lo que quieres es eliminar el primer elemento del array, utilizamos el método shift(). Este método también devuelve el elemento eliminado.

```
let colores = ["rojo", "azul", "verde"];  
let primerColor = colores.shift();  
  
console.log(colores);           // ["azul", "verde"]  
console.log(primerColor);       // "rojo"
```

En este ejemplo, hemos eliminado "rojo" del inicio del array.

4. unshift() – Añadir un elemento al principio del array

El método unshift() hace lo opuesto a shift(). Nos permite añadir uno o más elementos al principio de un array:

```
let colores = ["azul", "verde"];
colores.unshift("rojo");

console.log(colores); // ["rojo", "azul", "verde"]
```

Ahora, "rojo" ha sido añadido al inicio del array colores.

5. length – Conocer el tamaño del array

El tamaño de un array, es decir, cuántos elementos contiene, se puede saber utilizando la propiedad length. Esto es muy útil cuando necesitas trabajar con arrays de tamaño variable.

```
let animales = ["perro", "gato", "pájaro"];
console.log(animales.length); // 3
```

El array animales contiene 3 elementos, por lo que animales.length devolverá 3.

6. indexOf() – Encontrar el índice de un elemento

Si quieres saber en qué posición (índice) se encuentra un elemento dentro de un array, puedes utilizar el método indexOf(). Si el elemento no existe en el array, el método devuelve -1.

```
let dias = ["lunes", "martes", "miércoles", "jueves"];
let indice = dias.indexOf("miércoles");

console.log(indice); // 2
```

En este caso, "miércoles" está en el índice 2 del array dias.

7. slice() – Extraer una parte del array

El método slice() te permite copiar una parte del array sin modificar el original. Es muy útil cuando necesitas trabajar con una subsección de los datos.

```
let numeros = [1, 2, 3, 4, 5];
let subArray = numeros.slice(1, 4);

console.log(subArray); // [2, 3, 4]
console.log(numeros); // [1, 2, 3, 4, 5]
```

En este ejemplo, slice(1, 4) extrae los elementos desde el índice 1 hasta el índice 3 (sin incluir el índice 4).

8. join() – Unir los elementos de un array en una cadena de texto

El método join() convierte todos los elementos de un array en una sola cadena de texto, separándolos con el delimitador que elijas. Por defecto, utiliza una coma como separador.

```
let palabras = ["Hola", "cómo", "estás"];
let frase = palabras.join(" ");

console.log(frase); // "Hola cómo estás"
```

Aquí, hemos unido las palabras del array en una frase, utilizando un espacio como separador.

9. reverse() – Invertir el orden de los elementos

El método reverse() invierte el orden de los elementos en un array. Este método modifica el array original.

```
let numeros = [1, 2, 3, 4, 5];
numeros.reverse();

console.log(numeros); // [5, 4, 3, 2, 1]
```

Ahora los elementos de numeros están en orden inverso.

Ejemplos Prácticos

Ahora que has visto algunos de los métodos más comunes, veamos un ejemplo más complejo que combine varios de ellos:

```
let tareas = ["lavar platos", "hacer la cama", "sacar la basura"];

// Añadir una nueva tarea al final
tareas.push("estudiar JavaScript");

// Eliminar la última tarea y guardarla en una variable
let ultimaTarea = tareas.pop();

// Añadir una tarea al principio
tareas.unshift("desayunar");
```

```
// Saber cuántas tareas hay
let cantidadTareas = tareas.length;

console.log(tareas);           // ["desayunar", "lavar platos", "hacer
                                la cama", "sacar la basura"]
console.log(ultimaTarea);     // "estudiar JavaScript"
console.log(cantidadTareas);  // 4
```

Conclusión

Los arrays en JavaScript son una herramienta poderosa que te permite almacenar y manipular colecciones de datos de forma eficiente. Con los métodos comunes que acabamos de ver, puedes realizar una gran variedad de operaciones útiles, como añadir, eliminar, ordenar y buscar elementos. Además, son flexibles, ya que pueden contener cualquier tipo de datos.

Introducción a los Objetos en JavaScript

En JavaScript, un objeto es una estructura fundamental que nos permite almacenar y organizar información de una manera más compleja y significativa que con arrays o variables simples. Un objeto puede contener propiedades (que son básicamente pares clave-valor) y métodos (funciones que actúan sobre el propio objeto).

Piensa en un objeto como en un "contenedor" que almacena datos relacionados. Por ejemplo, un objeto puede representar un automóvil, una persona, un libro, etc. Cada una de estas entidades tiene características que podemos representar mediante propiedades.

Declaración de Objetos Literales

El método más sencillo y común para crear un objeto es utilizando la notación literal. Es una forma de escribir un objeto directamente dentro del código usando llaves {}.

```
let persona = {
  nombre: "Juan",
  edad: 30,
  profesion: "Ingeniero",
  saludar: function() {
    console.log("Hola, mi nombre es " + this.nombre);
  }
};
```

Aquí, hemos creado un objeto llamado persona con tres propiedades: nombre, edad y profesion. Además, tiene un método saludar, que es una función dentro del objeto que utiliza la palabra clave this para acceder a las propiedades del propio objeto.

Acceder a Propiedades y Métodos

Puedes acceder a las propiedades de un objeto utilizando el punto (.) o los corchetes ([]).

```
// Accediendo con el punto
console.log(persona.nombre); // "Juan"
console.log(persona.edad);   // 30

// Accediendo con corchetes
console.log(persona["profesion"]); // "Ingeniero"

// Llamando a un método
persona.saludar(); // "Hola, mi nombre es Juan"
```

Modificar Propiedades de un Objeto

También puedes modificar las propiedades de un objeto después de haberlo creado:

```
persona.edad = 31;
persona["profesion"] = "Desarrollador";

console.log(persona.edad);      // 31
console.log(persona.profesion); // "Desarrollador"
```

En este caso, cambiamos el valor de edad y profesion en el objeto persona.

Agregar Nuevas Propiedades o Métodos

Si quieres agregar una nueva propiedad o método a un objeto existente, puedes hacerlo fácilmente:

```
persona.nacionalidad = "Argentina";
persona.despedir = function() {
  console.log("Adiós, hasta luego");
};

console.log(persona.nacionalidad); // "Argentina"
persona.despedir();                // "Adiós, hasta luego"
```

Hemos añadido la propiedad nacionalidad y el método despedir() al objeto persona.

Función Constructora

Si quieres crear múltiples objetos con las mismas propiedades y métodos, una función constructora es una opción eficiente. Es como un "molde" que te permite crear varios objetos con características similares, pero con valores diferentes.

Para crear una función constructora, utilizamos la palabra clave function y luego el operador new para crear una nueva instancia del objeto.

```
function Coche(marca, modelo, anio) {
  this.marca = marca;
  this.modelo = modelo;
  this.anio = anio;
  this.detalles = function() {
    console.log("Coche: " + this.marca + " " + this.modelo + " ("
```

```

+ this.anio + "));
};
}

let coche1 = new Coche("Toyota", "Corolla", 2020);
let coche2 = new Coche("Honda", "Civic", 2021);

coche1.detalles(); // "Coche: Toyota Corolla (2020)"
coche2.detalles(); // "Coche: Honda Civic (2021)"

```

En este ejemplo, la función Coche actúa como un constructor. Creamos dos objetos (coche1 y coche2) a partir de esa función. Cada objeto tiene sus propios valores para marca, modelo, y año, pero comparte el mismo método detalles.

Clases en JavaScript (ES6)

A partir de ES6, se introdujeron las clases en JavaScript, que ofrecen una sintaxis más clara y ordenada para trabajar con objetos y funciones constructoras. Aunque las clases son básicamente una envoltura sobre las funciones constructoras, hacen que el código sea más legible y fácil de entender.

```

class Animal {
  constructor(nombre, especie) {
    this.nombre = nombre;
    this.especie = especie;
  }

  describir() {
    console.log(this.nombre + " es un " + this.especie);
  }
}

let perro = new Animal("Rex", "perro");
let gato = new Animal("Miau", "gato");

perro.describir(); // "Rex es un perro"
gato.describir(); // "Miau es un gato"

```

En este caso, Animal es una clase que tiene un constructor y un método describir(). Los objetos perro y gato son instancias de la clase Animal.

Métodos dentro de Clases

Puedes añadir más métodos dentro de las clases de forma similar a cómo lo harías en una función constructora:

```
class Coche {
  constructor(marca, modelo, anio) {
    this.marca = marca;
    this.modelo = modelo;
    this.anio = anio;
  }

  detalles() {
    console.log("Coche: " + this.marca + " " + this.modelo + " (" +
+ this.anio + ")");
  }

  arrancar() {
    console.log(this.marca + " ha arrancado.");
  }
}

let coche1 = new Coche("Tesla", "Model S", 2022);
coche1.detalles(); // "Coche: Tesla Model S (2022)"
coche1.arrancar(); // "Tesla ha arrancado."
```

Aquí hemos agregado el método arrancar() a la clase Coche.

El this

En JavaScript, el uso de this dentro de los objetos puede parecer confuso al principio, pero es una herramienta súper útil para que los objetos puedan "hablar de sí mismos". La palabra clave this dentro de un objeto se refiere a ese mismo

objeto. Es como si el objeto tuviera conciencia propia y pudiera acceder a sus propias propiedades o métodos.

Piensa en `this` como una forma en que el objeto puede decir: "Hey!, estoy hablando de mí mismo". Por ejemplo, si tienes un objeto como una persona, y esa persona tiene un método que saluda, cuando usas `this` dentro de ese método, estás diciendo: "Este soy yo, y esto es lo que quiero mostrar o hacer".

```
let persona = {  
  nombre: "Laura",  
  edad: 25,  
  saludar: function() {  
    console.log("Hola, soy " + this.nombre + " y tengo " +  
    this.edad + " años.");  
  }  
};
```

Aquí, cuando llamamos a `persona.saludar()`, la salida será:

```
Hola, soy Laura y tengo 25 años.
```

¿Por qué? Porque `this.nombre` dentro del método `saludar` le dice al objeto que se refiera a su propio valor de nombre, en este caso, "Laura". Lo mismo ocurre con `this.edad`, que hace referencia a la edad de ese mismo objeto, en este caso 25.

Es como si `this` fuera un espejo dentro del objeto. No importa si tienes varios objetos similares, cada vez que uses `this`, se referirá al objeto en particular que esté ejecutando el código en ese momento.

Otro ejemplo sencillo:

```
let coche = {  
  marca: "Toyota",
```

```
    modelo: "Corolla",
    arrancar: function() {
        console.log(this.marca + " " + this.modelo + " ha
arrancado.");
    }
};

coche.arrancar(); // "Toyota Corolla ha arrancado."
```

En este caso, `this.marca` y `this.modelo` permiten que el objeto `coche` acceda a sus propias propiedades `marca` y `modelo`. Sin `this`, no sabría a qué "coche" se refiere y no podríamos personalizar la acción.

En resumen, ***this*** es el truco mágico que hace que los objetos en JavaScript sepan de quién están hablando. Esto es especialmente útil cuando creamos muchos objetos y queremos que cada uno maneje su propio estado y comportamiento de manera independiente.

Ventajas de Usar Objetos

- **Organización:** Los objetos permiten organizar datos complejos de una manera lógica y estructurada.
- **Reutilización:** Con las funciones constructoras y clases, puedes crear múltiples objetos similares sin tener que duplicar el código.
- **Flexibilidad:** Los objetos pueden almacenar diferentes tipos de datos y métodos que interactúan entre sí.
- **Desarrollo Escalable:** Los objetos permiten que el código sea más fácil de mantener, ya que encapsulan datos y comportamientos en unidades independientes.

Ejemplos Prácticos

Ejemplo 1: Objeto de Persona

```
let persona = {
  nombre: "Ana",
  edad: 28,
  profesion: "Diseñadora",
  saludar: function() {
    console.log("Hola, soy " + this.nombre + " y tengo " +
this.edad + " años.");
  }
};

persona.saludar(); // "Hola, soy Ana y tengo 28 años."
```

Ejemplo 2: Función Constructora de Libro

```
function Libro(titulo, autor, año) {
  this.titulo = titulo;
  this.autor = autor;
  this.año = año;
  this.obtenerDetalles = function() {
    console.log("Libro: " + this.titulo + " por " + this.autor +
" (" + this.año + ")");
  };
}

let libro1 = new Libro("1984", "George Orwell", 1949);
libro1.obtenerDetalles(); // "Libro: 1984 por George Orwell
(1949)"
```

Ejemplo 3: Clase de Mascota

```
class Mascota {
  constructor(nombre, tipo) {
    this.nombre = nombre;
    this.tipo = tipo;
  }
}
```

```
    jugar() {  
        console.log(this.nombre + " está jugando.");  
    }  
}  
  
let miMascota = new Mascota("Firulais", "Perro");  
miMascota.jugar(); // "Firulais está jugando."
```

Conclusión

Los objetos en JavaScript son una parte esencial del lenguaje, ya que te permiten organizar, estructurar y manipular datos de manera eficiente. Con las funciones constructoras y clases, puedes crear objetos de manera más escalable y reutilizable. Las clases, introducidas en ES6, mejoran la legibilidad del código y hacen más intuitivo el trabajo con objetos.

Los objetos son fundamentales para el desarrollo en JavaScript, tanto en el front-end como en el back-end, así que isigue practicando para dominarlos!